

Rotte con "chiusure"

- Nei semplici casi visti finora, ogni rotta viene associata a un *callback* che è una "chiusura", funzione anonima che ci risparmia il disturbo di trovare un nome al callback!
- Esiste però un approccio alternativo per introdurre callback con nome
- Nell'esempio sopra, si ripete uno schema: ogni callback non fa altro che servire una *Page* (web), sostanzialmente statica (a meno dei parametri)
- Altre pagine da servire allungherebbero a dismisura la lista di *Route* e relativi callback/closure (qui brevi ma, in altri casi, potenzialmente lunghi)
- È più ordinato ricondurre tutti i callback che servono pagine a metodi di una classe che chiameremo *PageController* ("controller delle pagine")
- Si genera un (template, da riempire, di) **controller** con il tool *artisan*:

```
<!-- web.php -->

<?php
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return view('home'); });

Route::get('/contact', function () {
    return view('contact'); });

Route::get('/about', function () {
    return view('about'); });
```

```
my_app $ artisan make:controller PageController
Controller created successfully
```

PageController

- Codice PHP "boilerplate", generato prima da *artisan*
- In *web.php* in ogni route da definire, si inserisce come callback un metodo del *PageController* specificato con un array come qui a destra
- La route qui associa al path `'/contact'`, il metodo, della classe `PageController`, di nome `'contact'`

(Qui il metodo `contact()` di `PageController` fa ciò che faceva la closure soppiantata, cioè restituisce la view in *contact.blade.php*)

```
MY_APP
├── app
│   ├── Console
│   ├── Exceptions
│   └── Http
│       ├── Controllers
│       │   ├── Controller.php
│       │   └── PageController.php
│       ├── Middleware
│       └── Kernel.php
├── ...
└── ...

app > Http > Controllers > PageController.php

<?php
namespace App\Http\Controllers;

use Illuminate\Http\Request;

class PageController extends Controller
{
    // da definire (vedi sotto)
}

<!-- web.php -->
<?php
use Illuminate\Support\Facades\Route;

// la route /contact ha una closure come callback, commentiamoli via:
// Route::get('/contact', function () {return view('contacts'); });
// sostituendola con un callback metodo di un controller

use App\Http\Controllers\PageController;

Route::get('/contact', [PageController::class, 'contact']);

/* ora il callback è una funzione chiamata contact() che è un metodo del
controller PageController il cui codice di classe è in app/Http/Controllers */

class PageController extends Controller
{
    public function contact() {
        return view('contact');
    }
}
```

Route e Controller (risorse statiche)

- Il **controller** introdotto è stato quindi usato per:
 - riunire in una sola classe dei callback associati a
 - path riferibili a un gruppo omogeneo di "*risorse*" su cui opera la Web app
- Qui le **risorse** associate ai path e su cui operano i callback di *PageController* sono **pagine Web statiche** (le view servite dall'app ai clienti)
- Sono cioè risorse *stateless* o, se si vuole, con stato immutabile, o statico: si tratta di un caso particolare!
- I metodi di questo controller devono solo servire view statiche

```
<!-- web.php -->
<?php
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\PageController;

Route::get('/', [PageController::class, 'home']);
Route::get('/contact', [PageController::class, 'contact']);
Route::get('/about', [PageController::class, 'about']);
```

```
<?php
// file app/Http/Controllers/PageController.php

namespace app\Http\Controllers;
use Illuminate\Http\Request;

class PageController extends Controller
{
    public function home() {
        return view('welcome');
    }
    public function contact() {
        return view('contact');
    }
    public function about() {
        return view('about');
    }
}
```

Callback e *callable* nelle rotte

In generale, in Laravel il 2° elemento di una rotta, specifica la **reazione** dell'app al 1° (che è una richiesta HTTP)

Abbiamo visto due forme possibili per specificare tale **reazione**:

- a) le closure (funzioni anonime), v., nell'esempio, la rotta per **GET /**
- b) array di due elementi, v. nell'esempio, la rotta per **GET /path1**, cioè:

```
<!-- web.php -->
<?php
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\MyController;

Route::get('/', function () {return 'Ciao';} );
Route::get('/path1', [MyController::class, 'f']);
```

```
<?php
// file app/Http/Controllers/MyController.php
...
class MyController extends Controller
{
    public function f() { ... }
}
```

1. il nome della classe del controller, espresso come nell'esempio, tramite **MyController::class**
2. il nome del metodo del controller, come stringa, nell'esempio **'f'**

Tecnicamente, la **reazione** in una rotta dovrebbe essere specificata come un **callable** PHP, perché la "reazione" dell'app consiste nel "chiamarlo", cioè eseguirlo come **callback** quando arriva la richiesta HTTP individuata dalla rotta. Laravel raccomanda le forme (a) e (b) per specificare la **reazione** nelle rotte.

I *callable* nelle rotte Laravel: precisazioni

Per iniziare, ricordiamo che in PHP a oggetti `::class` è una *magic constant*

- restituisce a runtime il nome *fully-qualified* della classe come stringa, cioè `MyController::class` → `'App\Http\Controllers\MyController'`

```
use Illuminate\Support\Facades\Route;  
use App\Http\Controllers\MyController;  
  
Route::get('/', function () {return 'Ciao';} );  
Route::get('/path1', [MyController::class, 'f']);
```

Quindi, il callable `[MyController::class, 'f']` nella route per `'/path1'` produce lo stesso effetto di `['App\Http\Controllers\MyController', 'f']` e, in effetti, anche questo sarebbe un callable ammissibile (e non gli serve `use ...`)

Analogamente, è valida in PHP e Laravel (e non necessita di `use ...`), la rotta

```
Route::get('/path1', [\App\Http\Controllers\MyController::class, 'f'])
```

Ma è sempre consigliabile ricorrere alla forma breve, più leggibile, dichiarando `use ...` per ciascun controller menzionato nelle rotte

Attenzione: fino a Laravel 7, la route sarebbe stata specificata così (e senza `use ...`): `Route::get('/path1', 'MyController@f');`

Oggi, questa forma non è accettata, ma potreste trovarla in vecchi tutorial!

I *callable* nelle rotte Laravel: precisazioni

Per iniziare

```
use Illuminate\Support\Facades\Route;  
use App\Http\Controllers\MyController;  
  
Route::get('/', function () {return 'Ciao';} );  
Route::get('/path1', [MyController::class, 'f']);
```

cioè `MyController::class` → `'App\Http\Controllers\MyController'`

Quindi, il callable `[MyController::class, 'f']` nella route per `'/path1'` produce lo stesso effetto di `['App\Http\Controllers\MyController', 'f']` e, in effetti, anche questo sarebbe un callable ammissibile (e non gli serve `use ...`)

Analogamente, è valida in PHP e Laravel (e non necessita di `use ...`) , la rotta

```
Route::get('/path1', [\App\Http\Controllers\MyController::class, 'f'])
```

Ma è sempre consigliabile ricorrere alla forma breve, più leggibile, dichiarando

`use ...` per ciascun controller menzionato nelle rotte

Attenzione: fino a Laravel 7, la route sarebbe stata specificata così (e senza `use ...`):
`Route::get('/path1', 'MyController@f');`

Oggi, questa forma non è accettata, ma potreste trovarla in vecchi tutorial!

Route e Controller (risorse *stateful*)

- Come si è visto, un **controller** è una classe in cui raggruppare i metodi *callback*, associati a path riferibili a un gruppo omogeneo di "risorse" su cui opera la Web app
- Un controller risponde però anche a un altro intento, da cui prende il nome: **controllare** le risorse, leggendone e modificandone lo stato, cioè implementando la cosiddetta *business logic* della web app
- In generale, le risorse su cui opera la web app saranno **stateful**, cioè caratterizzate da uno stato **dinamico**, p.es. *User, Book, Car, Team ...* cui corrisponderanno i controller *UserController, BookController, CarController, TeamController ...*
- I metodi di tali controller non si limiteranno a servire una view, ma conterranno anche "business logic" di controllo della risorsa
P.es., per una risorsa *Book*, implementeranno la logica necessaria a trovare un libro, cancellare un libro, modificare un libro ..., cioè a leggere/modificare (get/set) lo **stato** della risorsa

Componenti Laravel finora

Finora abbiamo introdotto tre componenti software principali per Laravel:

- *routes*
- *views*
- *controllers*

Mancano ancora, soltanto, i...

- *models*

